VARIABLES AND DATATYPES

VARIABLE:

A named block of memory is called as a variable in Java. It acts like a container to store data.

Characteristics of a Variable:

1. To store and fetch data using a name.
2. Only one value can be stored in a variable.
3. Data stored in a variable is temporary.
4. Variable has a lifespan.
5. Variable has a scope.

Advantages of a Variable:

1. To store data
2. To fetch data
3. To re-assign data
4. To update data
5. To print data

Pre-requisite to create a variable:

In order to create a variable, we should know what type/kind of data we are about to store in that variable container. For that we use datatype. The datatype specifies what type of data is stored in the variable that we are creating or declaring.

STEP ONE – VARIABLE DECLARATION:

Declaration means creation in Java. The syntax to declare a variable is:

Syntax: **datatype identifier;**

Here, variable_name is given as the identifier. This statement in Java is called as Variable Declaration Statement. If we have to declare more than one variable of same datatype, separators are used.

Syntax: **datatype identifier1, identifier2, …….identifierN;**

What happens internally:

An empty memory block is created and named with the identifier that was given by programmer.

Eg: int a;

A variable of integer type is created with name a. **a** 

STEP TWO – VARIABLE INITIALIZATION:

We have created an empty memory block. But the purpose is to store data. Initialization means assigning a data/value to the variable for the first time. The syntax for initialization is:

Syntax: **identifier = data/value;**

This is also called as Assignment Statement as we use Assignment Operator (=) to provide a value to the variable container.

What happens internally:

The empty memory block is stored with provided data/value.

Eg: a = 10;

DECLARATION AND INITIALIZATION IN A SINGLE STEP:

The above two steps for Variable declaration and initialization can be done in a single step as follows:

Syntax: **datatype identifier = value/variable/expression;**

Eg: int b = 20;

What happens internally:

The empty memory block is created and immediately assigned with value that was assigned. It can also be done for more than one variable as follows:

Syntax:

datatype identifier1 = value1, identifier2 = value2, …….;

Example Program:

```
class ClassName
{
    public static void main (String [ ] args)
    {
        int a = 20; // store data
        int b = a; // fetch data
        a = 25; // re-assign data
        a = a + 10; // update data
        System.out.println(a); // print data
    }
}
```

DATATYPE:

Datatype is used during declaration of a variable. It is used to specify what type/kind of data to be stored in the variable when the variable container is created.

CLASSIFICATION OF DATATYPES:

1. Primitive datatypes
2. Non-Primitive datatypes

Primitive datatypes:

- They are pre-defined datatypes.
- They are used to create primitive variables.
- They are finite in number. They are totally 8 in number.
- They are keywords.
- They are stored in a single block of memory.
- Eg: byte, int, char, Boolean

Primitive datatypes have default value and default size. The following table provides information on the primitive datatypes.

Data Type	Default Value	Default size
boolean	false	1 bit
char	'\u0000'	2 byte
byte	0	1 byte
short	0	2 byte
int	0	4 byte
long	0L	8 byte
float	0.0f	4 byte
double	0.0d	8 byte

Primitive Data Types in Java

Number Literal has Integer which has 4 datatypes and Floating point which has 2 datatypes.

Integral datatypes:

byte, short, int, long

Floating point datatypes:

float, double

Character Literal has char datatype and Boolean Literal has boolean datatype.

Non-Primitive datatypes:

- They are user-defined datatypes.
- They are used to create non-primitive variables.
- They are infinite in number.
- They are not keywords.
- They are stored in multiple blocks of memory.
- Eg: ClassName, String, Array, Object reference

The default value of non-primitive datatypes is null.

The default size of non-primitive datatypes is non defined.

CLASSNAME AS A NON-PRIMITIVE DATATYPE:

All ClassNames in Java are Non-primitive datatypes.

Obviously, non-primitive variables can be created with ClassName datatype.

Eg: class ClassName

```
{
    public static void main ( String [ ] args)
    {
        ClassName cn;
        cn = null;
        System.out.println(cn);
    }
}
```

In the above example, non-primitive variable cn is created with ClassName non-primitive datatype. It is assigned to null.

TYPES OF VARIABLES

Variables are of two major classifications. They are:

Based on datatypes:

- Primitive variables
- Non-Primitive variables or Reference variables

Based on scope:

- Local variables
- Global variables, which are again of two types:
  - ❖ Static variables
  - ❖ Non-Static or Object or Instance variables

Primitive Variables:

The variables created with 8 Primitive datatypes are called as Primitive variables. The variables accept data/values only in that particular type they were created.

Eg: int a = 12;

char ch = 'Y';

Non-Primitive Variables:

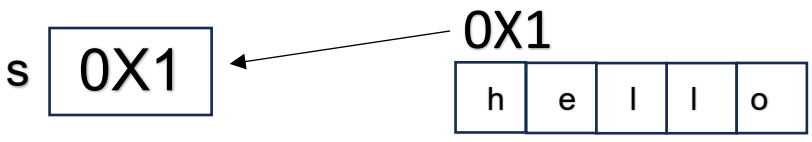
Suppose the programmer has to store the value "hello" in a variable. This group of characters cannot be stored in a single block of memory. So, continuous blocks of memory are required. This data can be stored in String datatype.

Eg: String s = "hello";

What happens internally:

Continuous blocks of memory space are allocated. But this is NOT named as s. Instead, an address which is of hexadecimal value is assigned to this memory by the processor.

And that address is stored in the non-primitive variable s.



Non-Primitive variable s is not used to store data/value. But it stores the address of another memory block. It can store address or null. Address is also called as Reference.

Hence, Non-Primitive variable is also called as Reference variable.

PROGRAM EXAMPLE FOR SCOPE:

```
class ClassName
    {
        int x;
        public static void main ( String [ ] args)
        {
            int a = 25;
        }
        int y;
    }
```

In the above example, three variables a, x and y are declared. The variable a is declared within the main method block, but the variables x and y are declared directly in the class block.

The variables x and y have Global scope whereas variable a has Local scope, which means it is a local variable of method block. The variable a can be accessed only within the method block.

Scope means visibility or accessibility of a variable.

LOCAL VARIABLE:

A variable declared in a method block or any other block (like initializer) but not directly in the class block is called as a Local Variable.

Characteristics of Local Variable:

1. A local variable must be initialized. There is no default value assigned to a local variable.
2. Local variable can be accessed only within the block where it was declared.
3. Two local variables cannot be declared with same name within the same scope.

PRIMITIVE DATATYPE	NON-PRIMITIVE DATATYPE
To declare primitive variables	To declare non-primitive variables
They are keywords	They are not keywords
They are pre-defined	They are not pre-defined(user-defined or custom for ClassName datatype)
They are finite (8 in number)	They are infinite
They are stored in single block of memory	They are stored in multiple blocks of memory
Eg: byte, short, int, double	Eg: Array, String, ClassName

TYPE CASTING:

The process of converting the datatype of a variable from one datatype to another datatype is called as Typecasting.

Primitive Typecasting:

The process of converting the datatype of a primitive variable from one primitive datatype to another primitive datatype is called as Primitive Typecasting.

Two types:

1. ***Widening***: The conversion of a primitive variable from lower range of primitive datatype to higher range of primitive datatype is widening. There is no data loss in widening.

   Eg: int a = 5;
   double d = a; // implicit widening

   Widening can be done both implicitly(by compiler) and explicitly(by programmer)
2. ***Narrowing***: The conversion of a primitive variable from higher range of primitive datatype to lower range of primitive datatype is Narrowing. There is a possibility of data loss in narrowing.

   Eg: double d = 12.45;
   int a = (int)d; // explicit narrowing

   Narrowing can be done only explicitly(by programmer)

   The round brackets() are called Typecast operator. The destination/target datatype is specified inside Typecast operator.

Non-Primitive Typecasting:

The process of converting the datatype of a non-primitive variable from one non-primitive datatype to another non-primitive datatype is called as Non-Primitive Typecasting.

Two types: Upcasting and Downcasting